

GSoC21 W6: LFortran ASR and Values

Rohit Goswami · 2021-07-16 · gsoc21 · fortran · ramblings

Enriching ASR nodes at compile time

Background

Serialized update for the [2021 Google Summer of Code](#) under the [fortran-lang](#) organization, mentored by Ondrej Certik.

Logistics

- Met with Ondrej on Monday and Wednesday
 - Discussed the design choices w.r.t. class hierarchies (and the lack thereof)

Overview

Note that the title is rather misleading, this post has nothing to do with the values of the LFortran project (which are by the way, fantastic), but instead is about adding more detail to the ASR nodes.

New Merge Requests

Implement `expr` values for Integer Binops (1045)

First example of using value-enriched nodes for operating during the `ast2asr` pass

Misc Logs

False Starts

Coming from a “throw it in a debugger” mindset, the ASR codebase and design of LFortran can be a bit disconcerting. This is because it is by design implemented mostly in terms of C structs which are cast explicitly throughout the code. Ondrej mentioned that the overhead of having a virtual function table for each instance of `LFortran::ASR` was prohibitive.

Why bother with such speed at the cost of the C++ guarantees?

Well for one thing, we are building a compiler, and one with an interactive kernel at that, the overhead must be kept low. No one wants to grab a coffee everytime they execute a `Python` jupyter cell and the goal should be the same for LFortran too.

Why not just add `value` to the `expr_t` structure?

This was a bit of a head scratcher to me, especially my debugger (`lldb`) offered no information on the value stored even in `Constant*` structures. The real solution, which required more of an understanding of the code (offered freely and kindly by my mentor) involves casting with `down_cast`.

Essentially, as far as the code base goes, we have a class based hierarchy, which we manipulate with `casts`.

Code duplication is a non-squinter though, since only `integer` and `real` have similar implementations, and Ondrej pointed out that over-engineering a design simply because two `if-esle` branches are similar is effectively yak shaving.

Conclusions

`real` was initially implemented as a string, which has now been replaced by Ondrej in 1066. This makes an extension to it trivial. I expect to focus on getting started on `functions` which will have the added benefit of easily adding functionality to the backend (since the `LLVM` backend can choose to simply use `value`). I intend to refocus my efforts towards getting `minidftatom` to work as well.